

TIPOS DE SISTEMAS

Sistemas Transaccionales

Definición

Un Sistema Transaccional (también conocido como TPS – Transaction Processing System) es un tipo de sistema de información diseñado para registrar, procesar y almacenar transacciones rutinarias de una organización.

Estas transacciones suelen ser operaciones básicas, repetitivas y de alto volumen, como ventas, compras, depósitos bancarios, cobros, inventarios, reservas, etc.

En pocas palabras, su función principal es automatizar procesos operativos y garantizar que los datos de las operaciones cotidianas se registren de manera rápida, precisa y segura.

Características principales

- **Alta frecuencia de uso** → Procesan miles o millones de transacciones diariamente.
- **Procesamiento en tiempo real o por lotes** → Dependiendo del sistema, se registran al instante o se acumulan para procesarse después.
- **Gran volumen de datos** → Manejan bases de datos robustas y en constante actualización.
- **Operación repetitiva y estructurada** → Se enfoca en tareas rutinarias que siguen reglas claras.

Características principales

- **Fiabilidad y seguridad** → Deben garantizar que los datos no se pierdan y mantener integridad (ejemplo: un depósito bancario no puede registrarse dos veces).
- **Velocidad y eficiencia** → Requiere procesar la información lo más rápido posible.
- **Integridad de datos (ACID)** → Cumplen con las propiedades de transacciones:
 - **Atomicidad** → Todo o nada.
 - **Consistencia** → No viola reglas del sistema.
 - **Isolamiento** → No interfiere con otras transacciones.
 - **Durabilidad** → Una vez hecho, permanece grabado.

Funciones principales

- **Captura de datos** → Entrada de información desde terminales, cajeros, sistemas POS, etc.
- **Procesamiento** → Validación, cálculo y registro de la transacción.
- **Almacenamiento** → Registro en bases de datos transaccionales.
- **Consulta** → Posibilidad de obtener reportes sobre las transacciones realizadas.
- **Integración con otros sistemas** → Alimentan sistemas de apoyo a decisiones (DSS) o sistemas estratégicos (ESS).

Ejemplos comunes

- **Banca** → Cajeros automáticos, transferencias, depósitos y retiros.
- **Comercio** → Puntos de venta (POS) para registrar compras.
- **Transporte** → Sistemas de reservaciones y emisión de boletos.
- **Gobierno** → Sistemas de recaudación de impuestos y registros civiles.
- **Salud** → Registro de pacientes, historial médico, facturación.
- **Educación** → Control de inscripciones, calificaciones, pagos escolares.

Ventajas

- Automatizan procesos rutinarios.
- Disminuyen errores humanos.
- Procesan grandes volúmenes de información de forma rápida.
- Mejoran la productividad y eficiencia de la organización.
- Garantizan exactitud en el registro de operaciones.
- Permiten trazabilidad (historial detallado de las transacciones).

Desventajas

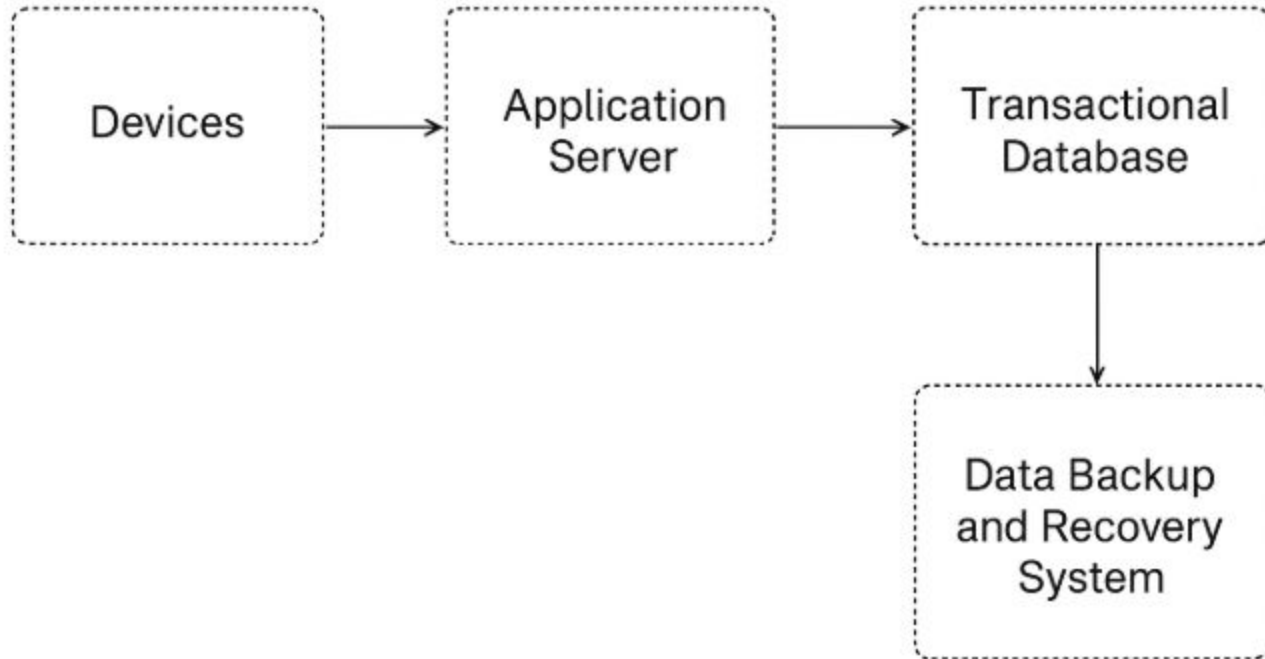
- Generalmente no apoyan directamente en la toma de decisiones estratégicas (son más operativos).
- Requieren infraestructura tecnológica robusta.
- Pueden ser costosos de implementar y mantener.
- Su rigidez hace difícil adaptarlos a cambios no planificados.
- Un fallo puede interrumpir operaciones críticas.

Arquitectura típica

Un sistema transaccional suele incluir:

- **Dispositivos de captura de datos** (terminales, POS, apps, cajeros).
- **Servidor de aplicaciones** que procesa las reglas de negocio.
- **Base de datos transaccional** optimizada para escritura y consultas rápidas.
- **Interfaces de usuario** simples y rápidas de usar.
- **Sistemas de respaldo y seguridad** para garantizar continuidad.

Arquitectura típica



Diferencia frente a otros sistemas

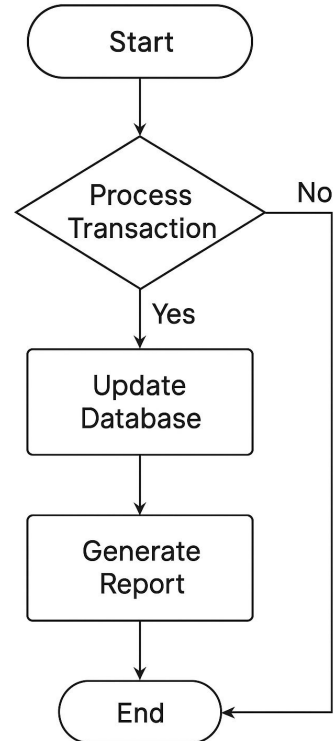
- **TPS (Transaccional)** → Operaciones diarias, rutinarias y repetitivas.
- **MIS (Management Information Systems)** → Reportes de gestión con base en datos transaccionales.
- **DSS (Decision Support Systems)** → Apoyo en decisiones, análisis y proyecciones.
- **ESS (Executive Support Systems)** → Información estratégica para directivos.
- El **TPS es la base** de los demás, ya que genera la información primaria sobre la cual se analizan tendencias y se toman decisiones.

Ejemplo práctico

Sistema de supermercado:

- Cliente pasa sus productos por el escáner.
- El sistema registra cada artículo, calcula el total y aplica descuentos.
- Al pagar, el sistema actualiza inventarios, registra la venta y genera ticket.
- La información se almacena en la base de datos, disponible para reportes diarios.

Ejemplo práctico



Sistemas de Toma de Decisiones

Definición

Un Sistema de Apoyo a la Decisión (DSS, por sus siglas en inglés) es una herramienta informática diseñada para ayudar a los directivos y profesionales a tomar decisiones basadas en datos, modelos y procesos de análisis.

No sustituye al ser humano en la toma de decisiones, sino que facilita el proceso ofreciendo información relevante, análisis y posibles escenarios.

Objetivo principal

- Mejorar la calidad de las decisiones.
- Reducir la incertidumbre en situaciones complejas.
- Apoyar en la evaluación de alternativas.
- Facilitar la toma de decisiones estructuradas, semiestructuradas o no estructuradas.

Características

- **Interactividad:** permite al usuario interactuar con los datos y modelos.
- **Flexibilidad:** se adapta a distintos problemas y estilos de decisión.
- **Orientado a decisiones:** no solo organiza datos, sino que genera alternativas y simulaciones.
- **Enfoque analítico:** emplea modelos matemáticos, estadísticos o de simulación.
- **Soporte a distintos niveles:** puede servir tanto a la alta dirección como a mandos intermedios.

Componentes de un DSS

Un sistema de apoyo a la decisión suele estar compuesto por:

- **Subsistema de datos**

- Base de datos interna (información de la empresa).
- Fuentes externas (mercado, competencia, estadísticas, big data).

- **Subsistema de modelos**

- Modelos matemáticos, financieros, de simulación, de optimización, etc.
- Permiten analizar escenarios y predecir resultados.

Componentes de un DSS

- **Subsistema de interfaz de usuario**
 - Pantallas, paneles de control (dashboards), reportes interactivos.
 - Es el medio de comunicación entre el usuario y el sistema.
- **Subsistema de conocimiento (opcional en DSS avanzados)**
 - Incluye reglas y heurísticas de expertos.
 - Se acerca a los sistemas expertos.

Tipos de DSS

- **DSS orientados a datos:** se centran en grandes volúmenes de información (ej. minería de datos).
- **DSS orientados a modelos:** se enfocan en el análisis mediante modelos matemáticos o de simulación.
- **DSS orientados al conocimiento:** incluyen inteligencia artificial o reglas de expertos.
- **DSS orientados a comunicaciones:** apoyan la colaboración en grupos (Group Decision Support Systems - GDSS).
- **DSS híbridos:** combinan varios enfoques anteriores.

Ejemplos de uso

- **Empresarial:** análisis de ventas, predicciones de demanda, decisiones de inversión.
- **Salud:** sistemas que sugieren diagnósticos o tratamientos.
- **Logística:** optimización de rutas de transporte.
- **Gobierno:** planeación urbana, políticas públicas.
- **Educación:** análisis del rendimiento estudiantil y predicción de deserción escolar.

Ventajas

- Facilita la evaluación de alternativas y escenarios hipotéticos.
- Aumenta la eficiencia en el proceso de decisión.
- Reduce el riesgo de errores humanos.
- Mejora la planeación estratégica.
- Integra datos de múltiples fuentes.

Desventajas

- Alto costo de implementación y mantenimiento.
- Requiere personal capacitado.
- No sustituye al juicio humano, solo lo complementa.
- Puede presentar sesgos si los datos están incompletos o mal estructurados.

Diferencia con otros sistemas

A diferencia de los Sistemas Transaccionales (TPS) que registran operaciones diarias, el DSS se centra en analizar información para tomar decisiones estratégicas o tácticas.

No es un Sistema Experto (ES) puro, aunque puede compartir características.

Tendencias actuales

- Big Data Analytics aplicado a DSS.
- Uso de Inteligencia Artificial y Machine Learning.
- Integración con Business Intelligence (BI).
- DSS en la nube (Cloud DSS).
- Visualización avanzada de datos (dashboards dinámicos).

REFERENCIAS

- Schneider, J., & Smalley, I. (2024, marzo 28). *What is a transaction processing system (TPS)?* IBM.
- Malyshev, A. (2025, 14 de abril). *What Is a Transaction Processing System: Definition, Types, and Benefits [Guide 2025]*. SDK.finance.
- Mahajan, R. (2009, 29 de diciembre). *Role of Transaction Processing System*. SSRN.
- UMSL. (s. f.). *Transaction Processing and Management Reporting Systems* [Capítulo]. Universidad de Missouri–St. Louis.
- Holsapple, C. W., & Whinston, A. B. (1996). *Decision Support Systems: A Knowledge-Based Approach*.
- Stanciu, C. O. (2009). *Decision Support Systems Architectures*.

Sistemas en Tiempo Real

Definición

Un sistema en tiempo real es aquel en el que la corrección del sistema no solo depende de los resultados lógicos de los cálculos, sino también del tiempo en que dichos resultados son entregados.

Es decir, además de producir resultados correctos, deben producirlos en el momento exacto en que se necesitan.

Un retraso, aunque sea mínimo, puede equivaler a un error de funcionamiento.

Características principales

- **Determinismo:** Capacidad de responder a eventos dentro de un intervalo de tiempo conocido y garantizado.
- **Confiabilidad:** Deben ser altamente seguros y con mínima probabilidad de fallo.
- **Disponibilidad:** Funcionan generalmente de forma continua (24/7).
- **Eficiencia:** Uso óptimo de recursos para evitar retardos.
- **Concurrencia:** Manejo simultáneo de múltiples procesos o eventos.
- **Priorización de tareas:** Se asigna importancia a procesos críticos sobre los secundarios.
- **Resiliencia:** El sistema debe recuperarse rápido frente a fallos parciales.

Clasificación

a) Tiempo real estricto (Hard Real-Time)

- El incumplimiento de una restricción temporal implica un fallo catastrófico.
- Ejemplo: Airbags en un automóvil, sistemas de control en aviones, sistemas médicos de soporte vital.

Clasificación

b) Tiempo real flexible (Soft Real-Time)

- Los plazos son importantes, pero se toleran retrasos ocasionales, aunque reducen el desempeño.
- Ejemplo: Streaming de video/audio, videojuegos en línea.

Clasificación

c) Tiempo real firme (Firm Real-Time)

- Un resultado entregado fuera del plazo es inútil, aunque no cause catástrofe.
- Ejemplo: Procesamiento de transacciones bancarias, control en sistemas industriales.

Ventajas

- Respuestas rápidas y garantizadas.
- Fiabilidad en sistemas críticos.
- Optimización de recursos de hardware y software.
- Control preciso en aplicaciones industriales o científicas.
- Soporte para operaciones simultáneas y multitarea.

Desventajas

- Complejidad en el diseño y programación.
- Costos de desarrollo e implementación elevados.
- Difícil de probar y validar (garantizar que siempre cumple tiempos límite).
- Requiere hardware especializado en muchos casos.

Ejemplos y aplicaciones

- **Aeronáutica:** sistemas de control de vuelo, radares, drones.
- **Automotriz:** frenos ABS, control de motores, bolsas de aire.
- **Salud:** monitores cardíacos, marcapasos.
- **Industria:** robots de ensamblaje, líneas de producción.
- **Telecomunicaciones:** gestión de redes, transmisión de datos en tiempo real.
- **Militar:** sistemas de defensa, radares.
- **Entretenimiento:** videojuegos en línea, streaming.

Funcionalidades básicas

- **Planificación (Scheduling):** asegura que las tareas críticas se ejecuten a tiempo (ej. algoritmos *Rate Monotonic* o *Earliest Deadline First*).
- **Sincronización:** coordinación de tareas para evitar conflictos de recursos.
- **Gestión de interrupciones:** manejo rápido y ordenado de señales externas.
- **Monitorización:** supervisión del sistema para garantizar que cumple plazos.

Ejemplos de software y sistemas operativos en tiempo real
(RTOS)

VxWorks

Desarrollado por Wind River Systems.

¿Qué hace?

Coordina tareas críticas en sistemas embebidos donde cada microsegundo cuenta.

Uso principal:

- Aeronáutica (naves espaciales de la NASA como Mars Rover).
- Telecomunicaciones.
- Automotriz (controladores en autos).
- Defensa y sistemas militares.



VxWorks

Desarrollado por Wind River Systems.

Características:

- Alta confiabilidad y seguridad.
- Soporte para multitarea y planificación determinista.
- Muy modular: se adapta a distintos dispositivos embebidos.



VxWorks

QNX Neutrino

Desarrollado por BlackBerry (anteriormente QNX Software Systems).

¿Qué hace?: Permite correr aplicaciones críticas en entornos industriales y automotrices con máxima tolerancia a fallos.

Uso principal:

- Automóviles (infotainment y control de seguridad).
- Energía (centrales eléctricas).
- Sistemas médicos.



QNX Neutrino

Desarrollado por BlackBerry (anteriormente QNX Software Systems).

Características:

- Arquitectura microkernel, que mejora la estabilidad (si un módulo falla, no colapsa todo el sistema).
- Soporte para multiprocesamiento simétrico (SMP).
- Certificaciones de seguridad industrial.



RTLinux / PREEMPT-RT (Linux en tiempo real)

Versión modificada de Linux que permite tiempo real estricto mediante el parche PREEMPT-RT.

¿Qué hace?: Convierte un sistema Linux convencional en un entorno capaz de responder en tiempo real.

Uso principal:

- Robots industriales.
- Sistemas de audio de baja latencia.
- Laboratorios de investigación.



RTLinux / PREEMPT-RT (Linux en tiempo real)

Versión modificada de Linux que permite tiempo real estricto mediante el parche PREEMPT-RT.

Características:

- Compatibilidad con aplicaciones Linux estándar.
- Reduce la latencia en interrupciones.
- Ideal para investigación y prototipado.



FreeRTOS

RTOS de código abierto, muy ligero, patrocinado actualmente por Amazon Web Services (AWS).

¿Qué hace?: Administra tareas en dispositivos pequeños con recursos muy limitados.

Uso principal:

- IoT (Internet of Things).
- Domótica (sensores, cerraduras inteligentes, termostatos).
- Pequeños sistemas embebidos.



FreeRTOS

RTOS de código abierto, muy ligero, patrocinado actualmente por Amazon Web Services (AWS).

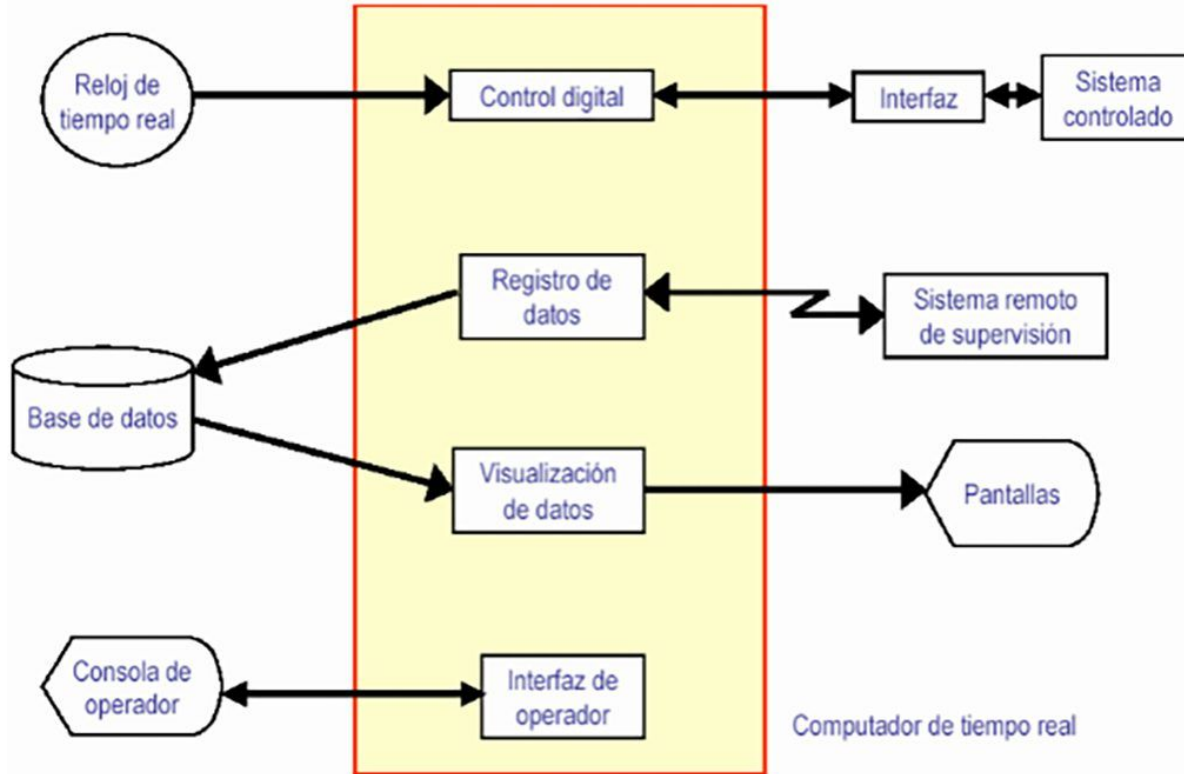
Características:

- Código fuente libre y portable.
- Bajo consumo de recursos (ideal para microcontroladores).
- Soporte para planificación de tareas con prioridades.



RTOS	Tipo	Enfoque principal	Usos típicos
VxWorks	Comercial	Alta confiabilidad y modularidad	NASA, autos, telecomunicaciones
QNX	Comercial	Microkernel, seguridad y estabilidad	Automóviles, industria, salud
RTLinux	Libre/Modificado	Linux en tiempo real	Robots, audio, investigación
FreeRTOS	Open Source	Ultra ligero, IoT y microcontroladores	IoT, sensores, domótica

Representación gráfica (visión general)



Referencias (APA)

- Burns, A., & Wellings, A. (2009). *Real-Time Systems and Programming Languages: Ada, Real-Time Java and Real-Time POSIX*. Addison-Wesley.
- Liu, J. W. S. (2000). *Real-Time Systems*. Prentice Hall.
- Kopetz, H. (2011). *Real-Time Systems: Design Principles for Distributed Embedded Applications*. Springer.
- Tanenbaum, A. S., & Bos, H. (2015). *Modern Operating Systems* (4th ed.). Pearson.

Sistemas Distribuidos

Definición

Un sistema distribuido es un conjunto de computadoras independientes que se presentan a los usuarios como un único sistema coherente. Estas computadoras, conocidas como *nodos*, cooperan entre sí para lograr un objetivo común, compartiendo recursos, información y poder de cómputo.

Características principales

- **Transparencia:** Ocultan la complejidad de la distribución a los usuarios. Ejemplos: transparencia de acceso, ubicación, replicación, concurrencia, fallos y migración.
- **Escalabilidad:** Capacidad de crecer sin degradar el rendimiento.
- **Concurrencia:** Varios procesos se ejecutan simultáneamente sobre distintos nodos.
- **Tolerancia a fallos:** Si un nodo falla, el sistema sigue funcionando.
- **Heterogeneidad:** Integran distintos hardware, sistemas operativos y redes.
- **Comunicación por mensajes:** La interacción entre nodos se da mediante paso de mensajes.

Ventajas

- Uso eficiente de recursos distribuidos.
- Alta disponibilidad y tolerancia a fallos.
- Escalabilidad horizontal (añadir más nodos).
- Posibilidad de cómputo paralelo y reducción de tiempos de procesamiento.
- Cercanía a los usuarios (reducción de latencia en servicios globales).

Desventajas

- Complejidad en el diseño y la programación.
- Problemas de seguridad y sincronización.
- Fallos parciales difíciles de detectar.
- Costos de comunicación elevados en comparación con sistemas centralizados.
- Requiere políticas de consenso para mantener la coherencia.

Modelos de Sistemas Distribuidos

- Cliente-Servidor: El cliente solicita servicios y el servidor responde.
- P2P (Peer-to-Peer): Todos los nodos actúan como clientes y servidores.
- Modelo de objetos distribuidos: Comunicación mediante invocación de métodos remotos.
- Modelos híbridos: Combinan cliente-servidor y P2P (ej. nube actual).

Ejemplos de uso

- Redes sociales: Facebook, Twitter, TikTok.
- Sistemas de archivos distribuidos: Google File System (GFS), HDFS (Hadoop).
- Bases de datos distribuidas: Cassandra, MongoDB, CockroachDB.
- Aplicaciones científicas: SETI@home, BOINC.
- Servicios en la nube: AWS, Azure, Google Cloud.

Componentes básicos

- Procesadores/Nodos: Computadoras que ejecutan procesos distribuidos.
- Red de interconexión: Generalmente TCP/IP.
- Middleware: Capa de software que proporciona comunicación y transparencia (ej. CORBA, gRPC).
- Protocolos de comunicación: RPC, RMI, sockets.

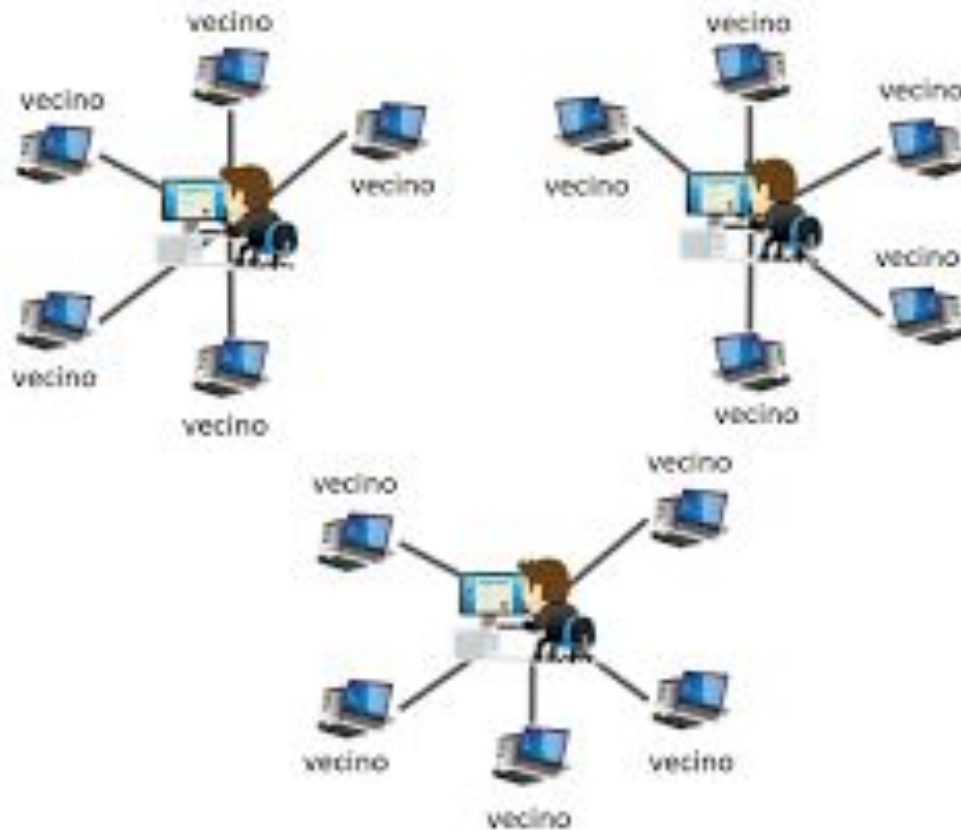
Problemas fundamentales

- Sincronización del tiempo: Relojes lógicos de Lamport, relojes vectoriales.
- Consistencia de datos: Modelos de consistencia (estricta, causal, eventual).
- Coordinación y consenso: Algoritmos como Paxos, Raft.
- Tolerancia a fallos bizantinos: Manejo de nodos maliciosos o con errores arbitrarios.

Aplicaciones modernas

- Blockchain y Criptomonedas: Bitcoin, Ethereum (altamente distribuidos y descentralizados).
- Microservicios: Arquitecturas distribuidas para aplicaciones empresariales.
- Edge Computing: Procesamiento cercano al usuario final.
- CDN (Content Delivery Networks): Akamai, Cloudflare.

Diagrama Ejemplo



Referencias

- Coulouris, G., Dollimore, J., Kindberg, T., & Blair, G. (2011). *Distributed systems: Concepts and design* (5th ed.). Addison-Wesley.
- Tanenbaum, A. S., & van Steen, M. (2017). *Distributed systems: Principles and paradigms* (2nd ed.). Pearson.
- Andrew S. Tanenbaum, & Herbert Bos. (2014). *Modern operating systems* (4th ed.). Pearson.
- George, C., & Shvartsman, A. A. (2007). *Introduction to reliable and secure distributed programming*. Springer.
- Attiya, H., & Welch, J. (2004). *Distributed computing: Fundamentals, simulations, and advanced topics* (2nd ed.). Wiley.